

Design and Analysis of (Backtest) Experiments

R/Finance 2023: From zero to a gazillion and back again

Brian G. Peterson
updated 18 May 2023

Introduction

- statistician R.A. Fisher (1935, 1926) pioneered “Design of Experiments”
- widely applied in fields such as
 - agriculture,
 - psychology studies,
 - pharmaceutical research,
 - manufacturing, and
 - chemical engineering
- *not* widely applied in quantitative finance

Problem and proposed solution

Problem:

- quantitative strategies often have lots of free parameters
- and multiple model components that interact in mysterious ways
- parameter optimization can take days or weeks on large clusters
- and has a high risk of overfitting

Why we should consider DoE:

- evaluate cause and effects of changes on inputs
- methodically test parameters to evaluate parameter dependencies and overcome noise
- overcome biases that assume data is randomized, independent and normally distributed
 - DoE enforces normalization of the parameter search space
 - DoE allows the researcher to more readily examine the i.i.d. assumption
- make better predictions of the effects of parameter changes
- perhaps most importantly, to save orders of magnitude of time/effort from an exhaustive brute force search

Word Salad

- Design of Experiments (DoE) is filled with difficult terminology
- the words themselves are not hard *but* they are drawn from the fields of research where they were employed
- “factors” in DoE are the free parameters of the model
- “contrasts” from biological sciences describe interactions between parameters
- “treatment plans” from medicine assess different parameter search spaces
- “blocks” and “plots” from agricultural research apply grouping and randomization of parameters to detect effects or minimize random or uncontrollable influences
- I’ll try to present the techniques using more common data science and quantitative descriptors

Test it All!

- *why not test all combinations of all parameters?*
- too many combinations, tens of thousands to millions to billions
- too expensive to test, in compute, in time, in money, in “wasted” output
- in any non-smooth or non-linear interaction, multiple testing bias is a huge problem (Harvey and Liu 2015; Bailey and López de Prado 2014)
- Statistical models like Bonferroni; Holm; BHY (Benjamini, Hochberg and Yekutieli) exist to test for and adjust for multiple testing bias
- also, with fixed level designs, the power and precision is increased by the product of the levels
- tl;dr; more parsimonious tests can actually have significantly *higher* power!

One at a Time

- many classical approaches to parameter experiments vary one variable at a time
- there are two main problems with this approach:
 - it is inefficient
 - it can't account for parameter interactions
- additionally, it assumes that you can hold constant or fixed *all* the other parameters

Small Example - Bollinger Bands

- band and midpoints strategies were first popularized by John Bollinger (2002) in the 1980's
- the basic idea is that you have some measure of fair value (the midpoint)
- and some measure of excess volatility (the bands)
- typically crossing the upper(lower) band is a sell(buy) signal as a measure of likely reversion towards the midpoint
- the simple example we will use here has two parameters
 - standard deviations (the bands) in a range of 1-3
 - moving average (the midpoint) from 10-40
- with a step size of 1, this creates 93 parameter combinations
- strategy is written in *quantstrat* and tested with *apply.paramset()*

Parameter (Factor) Importance

```
# fit linear models additive and multiplicative
```

```
m1 <- lm(Net.Trading.PL~MAparam+SDparam, data=ParamStats)
```

```
m2 <- lm(Net.Trading.PL~MAparam*SDparam, data=ParamStats)
```

```
m3 <- lm(Num.Trades~MAparam+SDparam, data=ParamStats)
```

```
m4 <- lm(Num.Trades~MAparam*SDparam, data=ParamStats)
```

	Net.Trading.PL		Num.Trades	
	(1)	(2)	(3)	(4)
MAparam	609.490 ^{***} (51.730)	998.720 ^{***} (130.216)	-3.292 ^{***} (0.247)	-8.223 ^{***} (0.336)
SDparam	-3,563.112 ^{***} (566.670)	1,302.263 (1,600.500)	-69.935 ^{***} (2.704)	-131.574 ^{***} (4.126)
MAparam:SDparam		-194.615 ^{***} (60.278)		2.466 ^{***} (0.155)
Constant	1,269.896 (1,780.733)	-8,460.854 ^{**} (3,457.474)	324.485 ^{***} (8.498)	447.761 ^{***} (8.913)
N	93	93	93	93
R ²	0.665	0.700	0.904	0.975
Adjusted R ²	0.657	0.690	0.902	0.974
Residual Std. Error	4,461.963 (df = 90)	4,245.236 (df = 89)	21.294 (df = 90)	10.943 (df = 89)
F Statistic	89.179 ^{***} (df = 2; 90)	69.152 ^{***} (df = 3; 89)	423.273 ^{***} (df = 2; 90)	1,152.358 ^{***} (df = 3; 89)

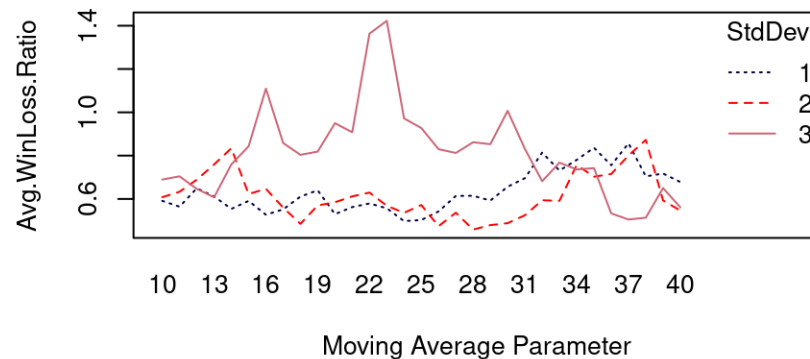
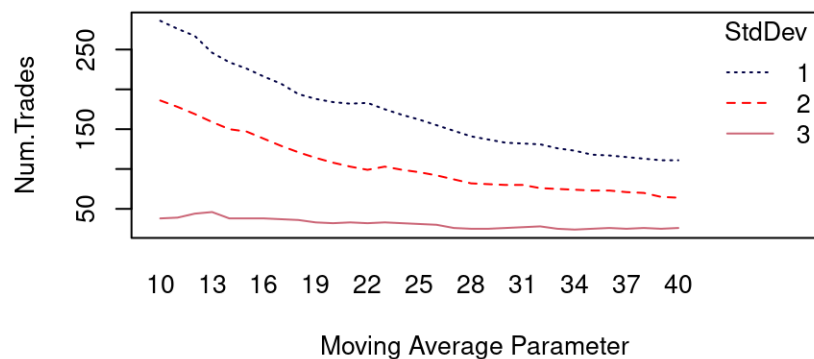
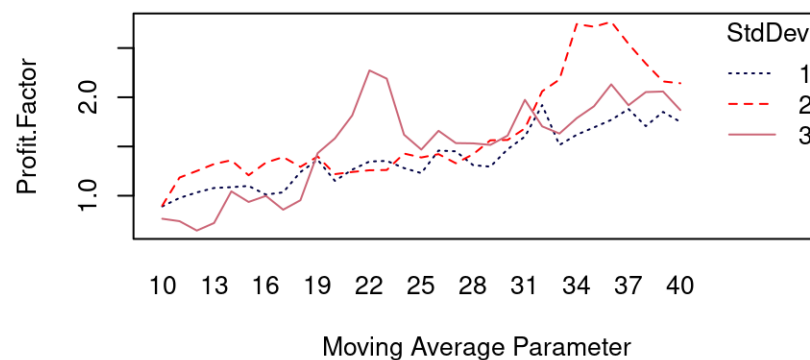
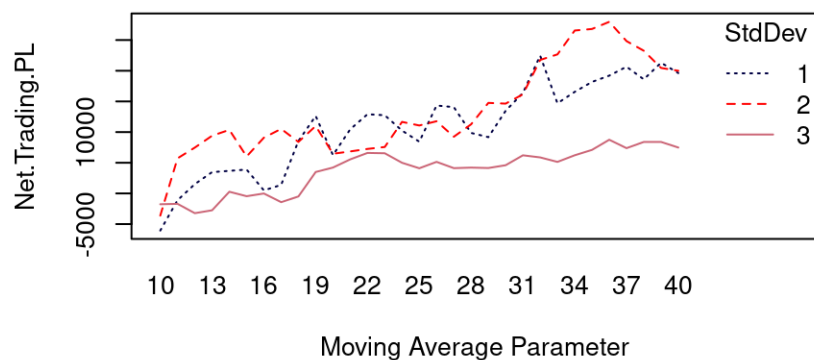
Notes:

^{***}Significant at the 1 percent level.

^{**}Significant at the 5 percent level.

^{*}Significant at the 10 percent level.

Parameter Interactions (cont.)



- *interaction.plot()* is a 'stats' base R function, and *tradeStats()* comes from *quantstrat*

Screening Designs

- screening designs exist to:
 - evaluate parameter importance
 - determine interactions between critical parameters
- if the interactions are deemed to be linear,
 - use high and low bounds for each parameter
 - 2^k combinations where k is number of parameters
- if the interactions are convex or quadratic
 - use 3 or 5 levels for each parameter
 - n^k combinations of n levels

Full Factorial Screening Designs

- in a “full factorial” (brute force) design, all combinations of all parameters are tested
- in R this is done using `expand.grid`:

```
threeway <- expand.grid(1:3,1:3,1:3,1:3,1:3)
```

```
fiveway <- expand.grid(1:5,1:5,1:5,1:5,1:5)
```

```
head(threeway)
```

```
##   Var1 Var2 Var3 Var4 Var5
## 1    1    1    1    1    1
## 2    2    1    1    1    1
## 3    3    1    1    1    1
## 4    1    2    1    1    1
## 5    2    2    1    1    1
## 6    3    2    1    1    1
```

- in python, you can use list comprehensions:

```
threeway = [(v,w,x, y, z) for v in range(3) for w in range(3) for x in range(3) for y in range(3) for z in range(3)]
```

```
threeway
```

```
## [(0, 0, 0, 0, 0), (0, 0, 0, 0, 1), (0, 0, 0, 0, 2), (0, 0, 0, 1, 0), (0, 0, 0, 1, 1), (0, 0, 0, 1, 2), (0, 0, 0, 2, 0), (0, 0, 0, 2, 1), (0, 0, 0, 2, 2), (0, 0, 1, 0, 0), (0, 0, 1, 0, 1), (0, 0, 1, 0, 2), (0, 0, 1, 1, 0), (0, 0, 1, 1, 1), (0, 0, 1, 1, 2), (0, 0, 1, 2, 0), (0, 0, 1, 2, 1), (0, 0, 1, 2, 2), (0, 0, 2, 0, 0), (0, 0, 2, 0, 1), (0, 0, 2, 0, 2), (0, 0, 2, 1, 0), (0, 0, 2, 1, 1), (0, 0, 2, 1, 2), (0, 0, 2, 2, 0), (0, 0, 2, 2, 1), (0, 0, 2, 2, 2), (0, 1, 0, 0, 0), (0, 1, 0, 0, 1), (0, 1, 0, 0, 2), (0, 1, 0, 1, 0), (0, 1, 0, 1, 1), (0, 1, 0, 1, 2), (0, 1, 0, 2, 0), (0, 1, 0, 2, 1), (0, 1, 0, 2, 2), (0, 1, 1, 0, 0), (0, 1, 1, 0, 1), (0, 1, 1, 0, 2), (0, 1, 1, 1, 0), (0, 1, 1, 1, 1), (0, 1, 1, 1, 2), (0, 1, 1, 2, 0), (0, 1, 1, 2, 1), (0, 1, 1, 2, 2), (0, 1, 2, 0, 0), (0, 1, 2, 0, 1), (0, 1, 2, 0, 2), (0, 1, 2, 1, 0), (0, 1, 2, 1, 1), (0, 1, 2, 1, 2), (0, 1, 2, 2, 0), (0, 1, 2, 2, 1), (0, 1, 2, 2, 2), (0, 2, 0, 0, 0), (0, 2, 0, 0, 1), (0, 2, 0, 0, 2), (0, 2, 0, 1, 0), (0, 2, 0, 1, 1), (0, 2, 0, 1, 2), (0, 2, 0, 2, 0), (0, 2, 0, 2, 1), (0, 2, 0, 2, 2), (0, 2, 1, 0, 0), (0, 2, 1, 0, 1), (0, 2, 1, 0, 2), (0, 2, 1, 1, 0), (0, 2, 1, 1, 1), (0, 2, 1, 1, 2), (0, 2, 1, 2, 0), (0, 2, 1, 2, 1), (0, 2, 1, 2, 2), (0, 2, 2, 0, 0), (0, 2, 2, 0, 1), (0, 2, 2, 0, 2), (0, 2, 2, 1, 0), (0, 2, 2, 1, 1), (0, 2, 2, 1, 2), (0, 2, 2, 2, 0), (0, 2, 2, 2, 1), (0, 2, 2, 2, 2)]
```

Full Factorial (Brute Force) Designs (cont.)

Rows in three-way design: 243

					2	2	2	1	1
Var1	Var2	Var3	Var4	Var5	3	2	2	1	1
1	1	1	1	1	1	3	2	1	1
2	1	1	1	1	2	3	2	1	1
3	1	1	1	1	3	3	2	1	1
1	2	1	1	1	1	1	3	1	1
2	2	1	1	1	2	1	3	1	1
3	2	1	1	1	3	1	3	1	1
1	3	1	1	1	1	2	3	1	1
2	3	1	1	1	2	2	3	1	1
3	3	1	1	1	3	2	3	1	1
1	1	2	1	1	1	3	3	1	1
2	1	2	1	1	2	3	3	1	1
3	1	2	1	1	3	3	3	1	1
1	2	2	1	1	1	1	1	1	1

Reducing the number of levels

- standard design of experiments methodology focuses on screening designs as described earlier
- this produces 2^k combinations for a min/max linear combination or n^k combinations for more levels n and parameters k
- this can rapidly lead to hundreds of thousands or even millions of trials
- reducing the number of levels is key. levels should have a center point for non-linear interactions to maximize the ability to design a more parsimonious design
- parameter constraints are also key (e.g. parameters $f > (g \& h)$) but note that this can create unbalanced or incomplete designs
- be careful that improper choice of min/max or constraints doesn't produce empty result sets, as this complicates analysis

Reducing the number of parameters

- use simple 2^k or 3^k screening experiment designs to fix and eliminate non-significant parameters
- utilize additional levels or search space only on parameters that are most important in screening designs
- parameter importance, parameter interaction, and response surface analysis on screening designs will typically show:
 - only a small number of parameters have major impacts on outputs (see Appendix)
 - (quick) fix (to a constant) the tertiary or insignificant parameters, they don't matter (much) anyway
- be aware that improperly fixing or removing parameters may lead to sub-optimal designs in other ways

you still have too many trials?

- when the number of combinations is too large to reasonably test the next thing to do is to use some sort of subset
- these designs are called 'fractional factorial' designs because they test a fraction of all the parameters ("factors")
- there are many ways to do this sampling:
 - random
 - easy, but has low statistical power, and may not have good coverage of the space
 - has the advantage of incorporating complex constraints
 - Latin hypercube designs - random design points within a range seek to improve this coverage (Viana 2016)
 - omit certain parameter combinations or ranges
 - may miss effects which are more important than they appeared in a screening design
 - hierarchical Bayesian models
 - very powerful, but typically use multiple rounds of experiments to refine the parameters (Overstall, Woods, and Adamou 2017)

Algorithmic Fractional Designs

- you can algorithmically find a fraction of the combinations that still maximizes coverage
- this is known as a D-optimal design
 - for maximizing the variance/covariance coverage of the “D”eterminant of the multidimensional matrix
- the Fedorov algorithm maximizes coverage by calculating support points in the multivariate ellipsoid space (Fedorov 2010; Wheeler 2022)
- additionally, Monte Carlo Fedorov designs can get very close to D-optimal designs where there are many parameters or complex constraints

Fractional Factorial Fedorov Example

```
require('AlgDesign',quietly = TRUE)
```

```
threeway_des <- optFedorov(~quad(.),data=threeway,approximate=TRUE, center=TRUE)
```

```
fiveway_des <- optFedorov(~quad(.),data=fiveway,approximate=TRUE, center=TRUE)
```

full factorial vs Fedorov fractional factorial
design size

x

three-level full factorial combinations	243
three-level Fedorov fractional factorial	202
five-level full factorial combinations	3125
five-level Fedorov fractional factorial	230

Large (Toy) Example

Strategy Introduction

- Design of Experiments is most useful when the total search space is huge
- two indicator models:
 - Bollinger Band
 - MACD
 - both use a linear model moving average (ImMA) for a more realistic and more responsive trend filter
- configured as a breakout strategy with signal process looking for *both*:
 - an upward[downward] trend (as detected by the MACD signal indicator) *and*
 - a breakout cross up[down] through the Bollinger Band
 - this could have been modeled as a constraint, but is several orders of magnitude more efficient models as a vectorized hybrid signal
- four total (two long/short entry and two long/short exit) rules:
 - attempt to enter trades on the up[down] breakout signals, *and*
 - exit trades when the market crosses back down[up] through the band
- this example will use the 9 US iShare sector stock indices for a larger more realistic portfolio

Strategy Parameters

The test strategy has five parameters and three constraints:

- the standard deviation parameter for the Bollinger Bands 1-3
- the duration of the ImMA for
 - the Bollinger Band, and
 - Bollinger Band MA 30-80
 - the MACD fast, slow and signal MA's
 - Fast MA 1-30
 - Slow MA same as Bollinger Band MA (30-80)
 - Signal MA 9-32
 - Bollinger Band MA == MACD slow MA
 - MACD Fast MA < MACD Slow MA

- MACD Signal MA < MACD Fast MA

for a total design complexity of:

- five parameters (with two set to be equal),
- four primary signals and two composite triggering signals,
- four rules,
- four parameter distributions (plus the equality constraint), and
- three additional parameter constraints

with default parameters, on one core, this:

- runs in under 30 seconds,
- consumes ~80 Degrees of Freedom (DoF),
- and retains over 99.8% total DoF

Full Factorial (Brute Force)

- the full factorial (brute force) design is over 110,000 combinations
- when we increment all the parameter ranges except standard deviation (1:3) by 2 rather than incrementing by 1, the number of combinations is about 20,000.
- on a 6-core/12-thread/64GB machine, this example will run out of RAM
- on a relatively typical 8-core/16-thread/128GB RAM workstation, the (~20k paramsets) brute force test takes about 20 hours to complete (only scales at 1/2 linear!)
- on a 24-core/48-thread/256GB RAM server, the brute force parameter search test takes about 6 hours
- “Real” strategies with more parameters, more assets, more rules, higher frequency data, or all of these could easily take (much) longer to analyze.

Impact on Degrees of Freedom

Are we overfitting?

Degrees of freedom report for strategy: multi.obj

Total market observations: 52029

Degrees of freedom consumed by strategy: 19458

Total degrees of freedom remaining: 32571

Pct. Degrees of Freedom: 62.60%

- ideally, you would retain >95% of total available degrees of freedom as one measure of the likelihood of avoiding overfitting
- we generously only consider 1 additional DoF to be used up by each parameter test
- the most conservative counting would lead to negative degrees of freedom

Fractional Factorial Design

Could a more parsimonious design help?

- an unconstrained Fedorov design returns about 900 parameter sets,
 - a 95% reduction from the full factorial example tested above,
 - and a 99% reduction from the original increment by 1 design
- when constraints are applied to the Fedorov design, there are 80 parameter sets remaining which satisfy the constraints
 - This is a >99.9% reduction from the original set,
 - a >99.5% reduction from the tested set,
 - and ~92% reduction from the full Fedorov design set.

Degrees of freedom report for strategy: constrained.Fedorov

Total market observations: 52029

Degrees of freedom consumed by strategy: 257

Total degrees of freedom remaining: 51772

Pct. Degrees of Freedom: 99.51%

Parameter Importance Regressions

fit linear models additive and multiplicative

```
fm1 <- lm(Net.Trading.PL~nFAST+nSLOW+nSIG+nSLOW*sd, data=ParamStats.fap)
```

```
fm2 <- lm(Net.Trading.PL~nFAST*nSLOW*nSIG*nSLOW*sd, data=ParamStats.fap)
```

```
fm3 <- lm(Profit.To.Max.Draw~nFAST+nSLOW+nSIG+nSLOW*sd, data=ParamStats.fap)
```

```
fm4 <- lm(Profit.To.Max.Draw~nFAST*nSLOW*nSIG*nSLOW*sd, data=ParamStats.fap)
```

	Net.Trading.PL		Profit.To.Max.Draw	
	(1)	(2)	(3)	(4)
nFAST	4.121 (36.049)	-650.316 (1,211.429)	0.0001 (0.005)	0.042 (0.173)
nSLOW	27.843 (17.373)	-92.463 (184.504)	0.005** (0.002)	0.047* (0.026)
nSIG	-127.287*** (40.028)	-189.712 (456.054)	-0.023*** (0.006)	0.040 (0.065)
sd	-1,545.106*** (394.514)	-9,629.542** (4,891.035)	-0.235*** (0.056)	-0.561 (0.698)
nFAST:nSLOW		8.851 (21.338)		-0.002 (0.003)
nFAST:nSIG		27.125 (42.624)		-0.0002 (0.006)
nSLOW:nSIG		3.331 (7.893)		-0.002 (0.001)

Parameter Importance Discussion

- we can see from the prior slide that most of the direct parameter effects are significant
- we can also see that the parameter interactions are generally not significant
- of course this analysis might be somewhat different on other response variables

Key Points:

- with 80 parameter sets from a constrained Fedorov design, we determined the regression coefficients with high significance
- the 'optimal' parameter combinations from the Federov design are very close to the brute force design, and would be good starting points for local optimization if required
- so instead of:
 - 20 hours on a fast multi-core server and relatively high chances of overfitting,
 - a much more parsimonious design which runs in minutes on a workstation can deliver significant results

Conclusion

- designed experiments can lower the size of the parameter space to be tested by 3+ orders of magnitude
- proper design can result in more insight from the experiment (more inferential power)
- “boiling the ocean” can easily lead to overfitting, and risks multiple testing bias
- simultaneously optimize multiple objectives in tests
- provide a framework for iteratively more complex experiment designs with incremental benefits
- Design of Experiments is a tool that should be in every analyst’s toolbox

Thank You

Resources

- I highly recommend the excellent *Design and Analysis of Experiments with R* by John Lawson (2014)
- or *Design and Analysis of Experiments* by Dean, Voss, and Draguljić (2017) for its huge breadth as well as SAS and R implementations
- *python* is harder, there are no authoritative texts that I know, but check *pyDOE* and *Pyomo.DOE* packages (Lee et al. 2023; Wang and Dowling 2022)
- the online book [Process Improvement Using Data](#) by Kevin Dunn contains a chapter on Design of Experiments in python
- The *AlgDesign* package in R contains the algorithmic Fedorov fractional factorial design functionality used in this presentation (Wheeler 2022)
- *Statistics for Experimenters* by Box, Hunter, and Hunter (2005) is the best classic text and also includes incredibly useful introductions to the core statistics that drive the utility of designed experiments

Appendix - Additional Topics

Parameter Interactions

- direct regression effects are denoted via an additive model of the form

$$y \sim A + B$$

- interaction effects are denoted via a multiplicative model of the type

$$y \sim A * B$$

- the parameter importance regression demonstrated that the parameters are significant both individually and in combination
- *but how do we understand those interactions?*

Response Surface Plots

- the parameter interactions in the plots above can also be done in a three dimensional surface
- plotting this surface is easy, using any plotting package,
 - using two parameters and a response variable
 - using a parameter and two response variables
 - using a composite coded parameter interaction and up to two response variables
- gaining insight from a response surface is harder

Response surface analysis (cont.)

- parameter interactions and response surfaces are, at their core, regression problems
- standard regression models can be used to evaluate these interactions
- additional tools become available if input variables are coded to standard levels (e.g. 1:10, -1 | 0 | 1, etc.) for both categorical and continuous inputs
- software packages like `rsm` in R or the response surface functions in SAS can help in this coding and analysis
- some complexities of this analysis involve finding the path of steepest ascent (gradient descent) and in using vertex and centroid designs to inform search segmentation

Coding Parameter Values

- coding the parameters into standard ranges
 - 0 | 1 for min/max designs
 - -1 | 0 | 1 for convex designs
 - numerically coding categorical variables
- can transform the parameter space into easier to optimize surfaces like contours and curves and hyper spheres with known dimensions
- some software methodologies require rescaled or recoded inputs
- some predictions will be wildly off even if the outputs are transformed or scaled back
- some care needs to be taken to assess the applicability of formal coding methods to your problem space

Testing Only the Most Important

- most experiments with a large number of parameters have only a few parameters that have a large effect on the outcome
- parameter interactions likewise decrease in impact as the number of interaction parameters increases
- direct (or main) effects of individual parameters are more likely to be important than two-parameter interactions
- this property is known as the *hierarchical ordering principle* (Lawson 2014 , p. 195)
- so you can construct designs which test the most important main effects and interactions
- this hierarchical experiment design approach isn't too useful in finance,
- or if the number of parameters and interactions is really large,
- but it is widely used in some fields,
- (see Lawson Ch. 6 for the process)

Randomization

- most texts on Design of Experiments spend a large amount of time discussing various randomization and replication methodologies
- the randomization or replication (or both) methodologies average hidden sources of variation across all the trials
- in computational experiments:
 - randomization is almost never necessary
 - replication may be useful if the model contains a stochastic, probabilistic, or other method that needs to converge based on some random or conditional calculations

Split Plot Designs

- in agricultural experiments, some variables can only be varied on the entire 'plot' of land
- other variables can be varied within the plot
- split plot designs exist to make compromises for testing hard to vary factors alongside factors that can be more easily varied with a plot
- care must be taken to make sure that the splits remain balanced across input parameters
- it is also relevant to note that some ANOVA statistics must be modified for correct output on split plot designs
- this isn't relevant to backtesting, where all the conditions are the same for all experiments performed in the same time range
- split plot designs could be relevant for 'testing in production' where the number of trials that can be run simultaneously is limited

Block Designs

- block designs are used to average noise or nuisance factors when you have variables with different inherent properties that are likely to affect the results
- in finance, asset class is an obvious blocking factor, since bonds, stocks, FX, crypto, etc tend to have very different properties
- so the choice of parameters is going to vary across blocks
- but inside a block, there should be room for valid inference
- if all parameter combinations from a brute force or fractional factorial design are tested on all blocks, you can use this to assess properties on a block by block basis, and look for commonalities of response across blocks
- in computational experiments, it is often possible to do several block slices after the fact, as long as the inputs are balanced across blocks.

Iterative Sequencing of Experiments

It often makes sense to consider your testing plan in an iterative way:

- *Screening Experiment*: two-level or three-level factorial design on a single instrument or block to identify main main parameters and drivers of the parameter response
- *Response Surface*: block factorial design on 3 or 5 levels and multiple blocks to identify overall response curvature of the main parameters
- *Parameter Optimization*: detailed parameter optimization using larger number of levels for the key parameters, multi-parameter interactions
- *Robust Parameter Designs*: variations including methodical study of sources of noise and variations in response (may be combined with the prior optimization stage)

Gain the most insight from each iterative stage, and correct problems earlier

Sources of Noise

- after WW II, a Japanese engineer, Genichi Taguchi, applied the ideas of W.E. Deming upstream in product design and called them *off-line quality control* (Lawson 2014, Ch. 12)
- he identified many sources of noise that can lead to reduced quality:
- *Deterioration noise*:
 - changes in the product or components over time that cause degradation of output quality
 - for trading strategies, this arises from stability of strategy parameters over time due to time series non-stationarity effects
 - parameters have a certain lifecycle, and parameter fit deteriorates over time
- *Environmental noise*:
 - noise factors which are external in the environment of the process under study
 - for trading strategies, market state factors (volatility, correlation, trend), and
 - physical environmental factors such as network latency and jitter
- *Variational Noise*:
 - difference in quality between individual outputs of a process
 - the primary focus of most manufacturing quality control literature

Robust Parameter Design

- identify parameters that jointly minimize the impacts of as many noise factors as possible
- varying factors that are controlled by the analyst at design time (called *control factors*)
- as well as measuring the impact of *environmental factors*
- often incorporates block designs and multi-objective optimization
- building robust parameter choices requires identifying:
 - parameter interactions and importance as described previously,
 - parameters that are as stable as possible across regimes and states,
 - *control factors* which could be set to adjust to environmental changes,
 - and *regimes* which could lead to different parameter choices
- overall, the goal is to have the most stable and robust parameters as possible, and
- know when you need to change parameters anyway because the environment has changed

Experiment Design for Feature Engineering

- most of the focus of Design of Experiments is on whole-system outcomes
- designed experiments are also often applied relatively late in the overall process flow
- it is possible to use these techniques much earlier in the process, at the feature engineering stage
- the major difference is one of target variables
- the objectives for experiments on a single indicator or feature are focused on
 - goodness of fit,
 - periodicity and persistence of the measured feature, and
 - prediction accuracy

Multi-Objective Optimization

- many times you are not interested in just one effect
- many real problems involve many objectives
- so choosing the most robust (out of sample) parameters also requires taking these multiple objectives into account
- full discussion of this topic is topic unto itself, but some approaches include
 - combined or weighted objectives
 - desirability functions
 - treating objectives as hidden variables in a Bayesian optimization
 - multi-objective global solvers like NSGA-III

References

- Bailey, David H, and Marcos López de Prado. 2014. "The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality." *Journal of Portfolio Management, Forthcoming*. <http://www.davidhbailey.com/dhbpapers/deflated-sharpe.pdf>.
- Bollinger, John. 2002. *Bollinger on Bollinger Bands*. McGraw Hill Professional.
- Box, George EP, J Stuart Hunter, and William G Hunter. 2005. *Statistics for Experimenters*. Second. Wiley Hoboken, NJ.
- Dean, Angela, Daniel Voss, and Danel Draguljić. 2017. *Design and Analysis of Experiments*. 2nd ed. Springer.
- Fedorov, Valerii. 2010. "Optimal Experimental Design." *Wiley Interdisciplinary Reviews: Computational Statistics* 2 (5): 581–89.
- Fisher, Ronald Aylmer. 1926. "The Arrangement of Field Experiments." *Journal of the Ministry of Agriculture of Great Britian* 503: 506–7.
- . 1935. *The Design of Experiments*. Oliver & Boyd, Edinburgh & London.
- Harvey, Campbell R., and Yan Liu. 2015. "Backtesting." SSRN. <http://ssrn.com/abstract=2345489>.
- Lawson, John. 2014. *Design and Analysis of Experiments with r*. Vol. 115. CRC press.
- Lee, A, M Baudin, M Christopoulou, Y Colette, and JM Martinez. 2023. *pyDOE: The Experimental Design Package for Python*. <https://github.com/tisimst/pyDOE>.
- Overstall, Antony, David Woods, and Maria Adamou. 2017. "Acebayes: An r Package for Bayesian Optimal Design of Experiments via Approximate Coordinate Exchange." *Journal of Statistical Software*. <https://www.jstatsoft.org/article/view/v095i13>.